

2026-05-06-p1-f15-subagent-team-design

Phase 1 / Feature 15 — Subagent Team

Date: 2026-05-06 **Status:** Approved (auto-approved per programme cadence) **Programme:** CLI-Agent Fusion — Phase 1 port from claude-code

1. Goal

Ship a real, end-to-end **subagent dispatch** system. F15 lets the parent agent fan out independent units of work to one or more subagents, each running its own (inner) agent loop with its own LLM call(s) and its own tool registry, then collect their results back as they stream in.

Three concrete user surfaces ship together:

1. **Agent tool** — `task` (the literal tool name claude-code uses, picked here for cross-agent prompt portability per Q4=B). Registered via `internal/tools/registry.go::ToolRegistry.registerAllTools()` lazily once the `SubagentManager` is wired in (mirrors F13's `SetLSPManager` and F14's `SetSandboxManager`). Arguments: `description` (string, required), `prompt` (string, required), `isolation` (string enum: "none" (default) or "worktree"; per Q2=B), `subagent_type` (optional string; selects a configured profile if any).
2. **Slash command** `/subagents` — `list` (alias of bare `/subagents`), `status [id]`, `kill <id>`. **No cobra subcommand** (Q5=B). The slash surface is read-only inspection + targeted termination.
3. **Hybrid execution model** (Q1=C) — by default subagents run as in-process goroutines with their own agent loop. When the agent passes `isolation: "worktree"`, the subagent is spawned as a **subprocess** (re-exec of the host binary with a sentinel env var, mirroring F14's helper-mode dispatch) running inside an F04-managed worktree directory.

Streaming aggregation (Q3=B) — the manager returns a `<-chan SubagentResult` immediately on dispatch; results land on the channel as each subagent completes; the channel is closed when the last subagent finishes. A small `WaitAll(<-chan SubagentResult) []SubagentResult` helper drains synchronously for the call sites that prefer the blocking style. Both modes are first-class.

The scope of F15 is **dispatch + execution + streaming aggregation + worktree isolation + slash inspection**. Auto-merge of worktree changes back to the parent branch, persistent subagent state across CLI sessions, subagent-to-subagent direct messaging, and per-subagent CPU/memory quotas are explicitly deferred to F15.5 (§8).

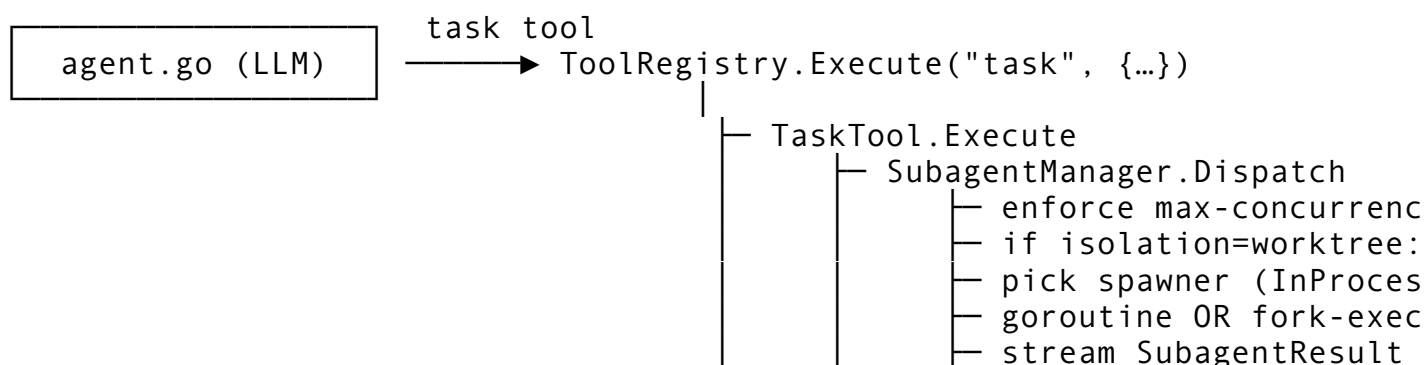
The subagent's inner agent loop is built from the same primitives the parent uses: `internal/llm.Provider` + `internal/tools.ToolRegistry` + a private session. Anti-bluff: a "subagent" that does not invoke the LLM, or that returns a hardcoded "I would do X" string, is a critical defect (§5.2).

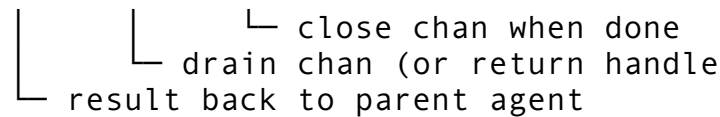
2. Architecture

Five layers, all under `internal/agent/subagent/`:

- `SubagentManager` — owns the lifecycle of running subagents. Its `Dispatch(ctx, *SubagentTask) (id string, results <-chan SubagentResult, err error)` is the only public dispatch path. Enforces (a) max-concurrency (default 5), (b) timeout (default 5min), (c) worktree creation via F04 when `isolation: "worktree"`, (d) kill-by-id from the slash command, (e) result streaming + channel close on terminal state.
- `SubagentSpawner interface` — two implementations:
 - `InProcessSpawner` — runs the subagent's inner agent loop in a goroutine. Each subagent gets its own (shallow-copied) `llm.Provider` reference and a fresh `ToolRegistry` view.
 - `SubprocessSpawner` — uses `os.Executable()` + a sentinel env var (`HELIX_SUBAGENT_INVOCATION=1`) to fork-exec the host binary. The child detects the env var at the very top of `main()` (F14 helper-mode pattern) and runs `RunAsSubagent()` instead of normal CLI dispatch. Stdout carries a JSON-encoded `SubagentResult`; the parent reads it and forwards to the caller's channel.
- `Subagent` — runtime record (ID, task, state, started/finished timestamps, isolation kind, worktree path if any, last error). State is `pending | running | succeeded | failed | killed | timed_out`.
- `TaskTool` — implements the `tools.Tool` interface (`Name/Description/Schema/Category/Validate/Execute`). Tool name: literal `task`. Category `CategorySubagent` (new constant; mirrors F13's `CategoryLSP` and F14's `CategorySandbox`). `Execute` calls `manager.Dispatch`, then either drains the channel (default, blocking) or returns the channel handle in the result map (when `streaming: true`). Default v1 behaviour is blocking-drain (one subagent per task call); the channel-handle return is the seam for a future `task_async` variant and is documented but kept behind a flag in v1.
- `SubagentsCommand` — `/subagents` slash command (`list/status/kill`). Mirrors F14's `/sandbox` command shape: defines a small `SubagentManager` interface in the `commands` package so the slash is testable with a fake while `main.go` passes the real `*subagent.SubagentManager`.

Worktree integration (per Q2=B): `SubagentManager.Dispatch` calls `worktree.Manager.EnterWorktree(ctx, name, baseBranch)` when `isolation == "worktree"`. The returned absolute path is set as the subprocess child's `cmd.Dir`. v1 does **not** auto-merge — subagent worktrees are returned to the user via the result map's `worktree_path` field for manual review using F04's existing `worktree cobra` command.





Subprocess child path (mirrors F14 helper-mode, see `internal/tools/sandbox/native_backend.go::IsHelperInvocation`):

```

main()
├─ if subagent.IsSubagentInvocation(): os.Exit(subagent.RunAsSubagent())
├─ if sandbox.IsHelperInvocation():    os.Exit(sandbox.RunAsHelper())
└─ ... normal CLI bootstrap

```

3. Components

3.1 New files

- `helix_code/internal/agent/subagent/types.go` — SubagentTask, SubagentResult, Isolation enum, SubagentState enum, SubagentSpawner interface, error sentinels, FakeLLMProvider (TEST PROVIDER, see §5.2).
- `helix_code/internal/agent/subagent/types_test.go`.
- `helix_code/internal/agent/subagent/inprocess_spawner.go` — goroutine-based spawner; runs an inner agent loop with the parent's `llm.Provider` (or a freshly-constructed one when configured) and a fresh `tools.ToolRegistry` view.
- `helix_code/internal/agent/subagent/inprocess_spawner_test.go` — uses FakeLLMProvider.
- `helix_code/internal/agent/subagent/subprocess_spawner.go` — fork-exec via `os.Executable() + HELIX_SUBAGENT_INVOCATION=1`; reads JSON-encoded SubagentResult from child stdout.
- `helix_code/internal/agent/subagent/subprocess_spawner_test.go` — uses an injectable Executor seam to assert argv + env.
- `helix_code/internal/agent/subagent/manager.go` — SubagentManager with dispatch + streaming aggregation + max-concurrency + kill-by-id.
- `helix_code/internal/agent/subagent/manager_test.go`.
- `helix_code/internal/agent/subagent/worktree_integration.go` — wraps F04's `worktree.Manager.EnterWorktree`; on dispatch failure, no worktree is created.
- `helix_code/internal/agent/subagent/worktree_integration_test.go` — runs against a real `git init`-ed tempdir.
- `helix_code/internal/agent/subagent/helper_mode.go` — `IsSubagentInvocation() + RunAsSubagent() int`. Helper-mode does NOT depend on the rest of the CLI bootstrap; it constructs a minimal subagent runtime from env-var-encoded payload, runs the inner agent loop, prints JSON to stdout, exits.
- `helix_code/internal/agent/subagent/helper_mode_test.go`.
- `helix_code/internal/tools/task_tool.go` — TaskTool implementing `tools.Tool` interface; tool name task.
- `helix_code/internal/tools/task_tool_test.go`.
- `helix_code/internal/commands/subagents_command.go` — `/subagents slash (list/status/kill)`.
- `helix_code/internal/commands/subagents_command_test.go`.
- `helix_code/tests/integration/subagent_test.go` — `//go:build integration, gated per §5.2`.

- `helix_code/tests/integration/cmd/p1f15_challenge/main.go` — runtime evidence harness.
- `challenges/p1-f15-subagent-team/CHALLENGE.md + run.sh`.

3.2 Modified files

- `helix_code/internal/tools/registry.go` — add `CategorySubagentToolCategory = "subagent".Add SetSubagentManager(m *subagent.SubagentManager)` (mirrors F13's `SetLSPManager` and F14's `SetSandboxManager`); the method lazily registers `TaskTool` so unit tests of the registry that don't supply a manager don't see the task tool.
- `helix_code/cmd/cli/main.go` — three lines:
 1. **First statement of `main()`:** `if subagent.IsSubagentInvocation()` { `os.Exit(subagent.RunAsSubagent())` }. MUST precede the existing `sandbox.IsHelperInvocation()` check.
 2. After the existing tool-registry construction: build a `SubagentManager`, call `registry.SetSubagentManager(mgr)`.
 3. Slash-command registration:
`slashRegistry.Register(commands.NewSubagentsCommand(mgr)).`

3.3 Types

```
// internal/agent/subagent/types.go

type Isolation string
const (
    IsolationNone      Isolation = "none"           // default; in-
        process goroutine
    IsolationWorktree Isolation = "worktree"       // F04 worktree +
        subprocess
)

type SubagentState string
const (
    StatePending      SubagentState = "pending"
    StateRunning      SubagentState = "running"
    StateSucceeded    SubagentState = "succeeded"
    StateFailed       SubagentState = "failed"
    StateKilled       SubagentState = "killed"
    StateTimedOut     SubagentState = "timed_out"
)

// SubagentTask is what the parent agent's `task` tool call
// builds.
type SubagentTask struct {
    Description string `json:"description"` //
        human-readable; logged at INFO
    Prompt      string  `json:"prompt"`      // the
        LLM prompt body; NEVER logged at INFO (CONST-042)
```

```

Isolation      Isolation      `json:"isolation"`          //
    default IsolationNone
SubagentType   string
    `json:"subagent_type,omitempty"` // optional profile
    selector
Timeout        time.Duration `json:"timeout"`          //
    default 5min, hard ceiling 30min
BaseBranch     string
    `json:"base_branch,omitempty"` // worktree base; empty →
    current HEAD
}

```

// SubagentResult is what each subagent emits on the streaming channel.

```

type SubagentResult struct {
    ID            string      `json:"id"`
    State         SubagentState `json:"state"`
    Output        string
    `json:"output"`          // LLM text response or
    aggregated tool output
    Error         string      `json:"error,omitempty"`
    ExitCode      int         `json:"exit_code"`          // 0
    success; only meaningful for subprocess
    StartedAt     time.Time   `json:"started_at"`
    FinishedAt    time.Time   `json:"finished_at"`
    Duration      time.Duration `json:"duration"`
    Isolation     Isolation   `json:"isolation"`
    WorktreePath  string
    `json:"worktree_path,omitempty"` // non-empty for
    worktree isolation
    ToolCalls     int         `json:"tool_calls"`          //
    number of tool invocations the subagent made
}

```

```

type SubagentSpawner interface {
    Spawn(ctx context.Context, sub *Subagent, sink chan<-
        SubagentResult) error
    Kind() string // "in-process" or "subprocess"
}

```

```

var (
    ErrMaxConcurrency          = errors.New("subagent: max
        concurrency reached")
    ErrSubagentNotFound       = errors.New("subagent: id not
        found")
    ErrWorktreeUnavailable   = errors.New("subagent: worktree
        manager not configured")
)

```

```

    ErrInvalidIsolation      = errors.New("subagent: isolation
        must be 'none' or 'worktree'")
    ErrSubagentTimeout        = errors.New("subagent: timed out")
    ErrSubagentKilled         = errors.New("subagent: killed via /
        subagents kill")
)

// internal/agent/subagent/manager.go

type SubagentManagerOptions struct {
    LLMProvider      llm.Provider          // required; the
        parent's provider
    ToolRegistry     *tools.ToolRegistry          // required;
        subagent tools are a fresh view of this registry
    WorktreeManager *worktree.Manager          // optional;
        required only when IsolationWorktree is requested
    InProcess        SubagentSpawner      // injectable for
        tests; default NewInProcessSpawner(...)
    Subprocess       SubagentSpawner      // injectable for
        tests; default NewSubprocessSpawner(...)
    MaxConcurrency   int                                // default 5
    DefaultTimeout   time.Duration          // default 5min,
        hard ceiling 30min
    Logger           *zap.Logger                        // optional; nil →
        zap.NewNop()
    Now              func() time.Time                  // injectable;
        default time.Now
}

type SubagentManager struct {
    opts      SubagentManagerOptions
    mu        sync.RWMutex
    running   map[string]*Subagent          // id → record
    sem       chan struct{}                 // bounded by
        MaxConcurrency
}

func NewSubagentManager(opts SubagentManagerOptions)
    (*SubagentManager, error)
func (m *SubagentManager) Dispatch(ctx context.Context, t
    *SubagentTask) (id string, results <-chan
    SubagentResult, err error)
func (m *SubagentManager) WaitAll(results <-chan SubagentResult)
    []SubagentResult          // synchronous helper
func (m *SubagentManager) List()
    []SubagentInfo              //
    for /subagents list

```

```

func (m *SubagentManager) Status(id string) (SubagentInfo,
    error) // for /subagents status
func (m *SubagentManager) Kill(id string)
    error // for /
    subagents kill
func (m *SubagentManager) Close(ctx context.Context)
    error // drains running,
    returns when all stopped or ctx done

type Subagent struct {
    ID          string
    Task        *SubagentTask
    State       SubagentState
    StartedAt   time.Time
    FinishedAt   time.Time
    WorktreePath string
    cancel       context.CancelFunc
    spawner     SubagentSpawner
    mu          sync.RWMutex
}

type SubagentInfo struct {
    ID          string
    Description  string
    State       SubagentState
    Isolation   Isolation
    WorktreePath string
    StartedAt   time.Time
    Duration    time.Duration // 0 if still running
}

// internal/tools/task_tool.go

type TaskTool struct {
    manager *subagent.SubagentManager
}

func NewTaskTool(m *subagent.SubagentManager) *TaskTool
func (t *TaskTool) Name() string { return "task" }
func (t *TaskTool) Description() string // documents
    description/prompt/isolation/subagent_type
func (t *TaskTool) Schema() ToolSchema // see below
func (t *TaskTool) Category() ToolCategory // CategorySubagent
func (t *TaskTool) Validate(params map[string]interface{}) error
func (t *TaskTool) Execute(ctx context.Context, params
    map[string]interface{}) (interface{}, error)

```

Tool result is a map[string]interface{} with keys id, state, output, error, duration_ms, tool_calls, worktree_path (only present for worktree isolation),

isolation. Default v1 behaviour is to drain a single-result channel and return the consolidated map.

3.4 User surfaces

Agent tool call:

Tool name	Schema	Returns
task	{description: string, prompt: string, isolation?: "none" "worktree" (default "none"), subagent_type?: string, timeout_seconds?: int (default 300; max 1800)}	{id, state, output, error?, duration_ms, tool_calls, isolation, worktree_path?}

Slash command /subagents: - /subagents (alias of list) — table: ID STATE ISOLATION DURATION DESCRIPTION. Killed/finished rows are kept for a short retention window (opts.RetentionAfterFinish, default 10min) so the user can inspect post-mortem. - /subagents status <id> — full record (state, isolation, worktree path if any, started_at, finished_at, error message if any). Output is **never** the prompt body (CONST-042) — the description field is shown instead. - /subagents kill <id> — calls manager.Kill(id). For in-process subagents this cancels the context; for subprocess subagents it sends SIGTERM (then SIGKILL after 5s if the child is still alive).

No cobra subcommand (Q5=B). Inspection and termination both go through the slash command.

3.5 Existing-code constraints

internal/agent/ already has its own Agent interface, BaseAgent type, AgentRegistry, etc. F15 deliberately does **not** modify those types. The subagent’s inner loop reuses BaseAgent.SetLLMProvider + BaseAgent.SetToolRegistry (already public per base_agent.go) to construct a fresh agent without touching the parent’s registry of agents. The SubagentManager and the existing AgentRegistry are intentionally separate — AgentRegistry is the long-lived “what agent types exist” registry, while SubagentManager is the per-task “what is currently running” registry. This separation keeps F15 additive.

internal/tools/registry.go already defines a Tool interface — TaskTool lives in package tools (NOT in subagent) to avoid a circular import, and imports the subagent package directly. This mirrors F13’s tool placement (LSPTool lives in tools, not in a subpackage), keeping the registry self-contained.

4. Data flow

4.1 Startup wiring (cmd/cli/main.go)

```
main()
├─ if subagent.IsSubagentInvocation(): os.Exit(subagent.RunAsSubagent())
├─ if sandbox.IsHelperInvocation(): os.Exit(sandbox.RunAsHelper())
├─ ... existing CLI bootstrap (cobra, config load, providers, ...)
├─ wtMgr := worktree.NewManager(repoRoot)
├─ saMgr, err := subagent.NewSubagentManager(subagent.SubagentManagerOpt{
│   LLMProvider: provider, ToolRegistry: toolReg,
│   WorktreeManager: wtMgr, MaxConcurrency: 5, DefaultTimeout: 5*time
│ })
├─ if err != nil: log WARN; saMgr = nil → registry will not register `t
├─ if saMgr != nil: toolReg.SetSubagentManager(saMgr)
├─ slashRegistry.Register(commands.NewSubagentsCommand(saMgr))
├─ defer saMgr.Close(ctx)
```

If `saMgr == nil` (e.g. `WorktreeManager` construction failed AND `IsolationWorktree` was passed) — the registry skips registering task, so the agent gets a clear “tool not found” error rather than a silent no-op. The `/subagents slash` still registers and prints subagent manager unavailable.

4.2 In-process dispatch flow

```
TaskTool.Execute(ctx, params)
├─ task := buildTask(params) // parse description/prompt/isolati
├─ id, results, err := mgr.Dispatch(ctx, task)
├─ if err != nil: return error
├─ collected := mgr.WaitAll(results) // drain until channel close
├─ return resultMap(collected[0])
```

```
SubagentManager.Dispatch(ctx, task)
├─ acquire sem (block up to ctx.Deadline) OR return ErrMaxConcurrency
├─ id := uuid.NewString()
├─ subCtx, cancel := context.WithTimeout(ctx, task.Timeout)
├─ sub := &Subagent{ID: id, Task: task, State: StatePending, cancel: can
├─ if task.Isolation == IsolationWorktree:
│   path, err := wtMgr.EnterWorktree(subCtx, id, task.BaseBranch)
│   if err: release sem; cancel; return err
│   sub.WorktreePath = path
│   sub.spawner = m.opts.Subprocess
├─ else:
│   sub.spawner = m.opts.InProcess
├─ register sub in m.running
├─ ch := make(chan SubagentResult, 1)
├─ go func() {
│   defer close(ch); defer m.release(id)
│   err := sub.spawner.Spawn(subCtx, sub, ch)
│   if err && no result was emitted: ch <- failedResult(sub, err)
│ }()
├─ return id, ch, nil
```

```
InProcessSpawner.Spawn(ctx, sub, sink)
├─ provider := sub.spawner.opts.LLMProvider
├─ inner := buildInnerAgent(provider, sub.spawner.opts.ToolRegistry) //
├─ start := now()
├─ resp, err := inner.Run(ctx, sub.Task.Prompt) // ACTUALLY invoke
├─ result := SubagentResult{ID: sub.ID, Output: resp.Text, State: stateF
├─ sink <- result
```

4.3 Subprocess (worktree-isolated) dispatch flow

```
SubprocessSpawner.Spawn(ctx, sub, sink)
├─ exe, _ := os.Executable()
├─ payload := encodeSubagentPayload(sub.Task, sub.WorktreePath, sub.ID)
├─ cmd := exec.CommandContext(ctx, exe)
├─ cmd.Dir = sub.WorktreePath // child run
├─ cmd.Env = append(os.Environ(),
    "HELIX_SUBAGENT_INVOCATION=1",
    "HELIX_SUBAGENT_PAYLOAD="+payload,
  )
├─ var stdout bytes.Buffer
├─ cmd.Stdout = &stdout
├─ cmd.Stderr = (a logger-prefixing writer)
├─ err := cmd.Run()
├─ result := decodeSubagentResult(stdout.Bytes())
├─ if decode fails: result = failedResult(sub, fmt.Errorf("subagent stdo
├─ result.WorktreePath = sub.WorktreePath
├─ result.Isolation = IsolationWorktree
├─ sink <- result

// In the child process:
RunAsSubagent() int
├─ payload := decodeSubagentPayload(os.Getenv("HELIX_SUBAGENT_PAYLOAD"))
├─ provider := buildProviderFromEnv() // F12: same
├─ toolReg := buildMinimalToolRegistry(provider) // shell, re
├─ inner := buildInnerAgent(provider, toolReg)
├─ result, err := inner.Run(ctx, payload.Prompt)
├─ json.NewEncoder(os.Stdout).Encode(SubagentResult{...})
├─ return 0 on success, 1 on err
```

The child does NOT register the task tool itself (preventing infinite subagent recursion in v1).
v1 cap: subagent depth = 1. Documented in §8.

4.4 Streaming + WaitAll

```
results := <-chan SubagentResult // returned by Dispatch
go func() { ... eventually close(results) }() // closed by manager goroutine

// Sync drain:
all := mgr.WaitAll(results) // blocks until close

// Async fanout (e.g., when the parent dispatches N subagents and merges):
for r := range results { handle(r) }
```

WaitAll is implemented as `for r := range ch { out = append(out, r) }` — trivial but documented as part of the public API so call sites don't need to remember the channel-iteration idiom.

4.5 Worktree integration (per Q2=B)

When `task.Isolation == IsolationWorktree`: - The manager calls `wtMgr.EnterWorktree(ctx, sub.ID, task.BaseBranch)`. - The worktree directory is `<repoRoot>/helix-worktrees/<sub.ID>/`. F04 already creates the parent dir. - **Parent's uncommitted changes**: the F04 `EnterWorktree` path uses `git worktree add <path> <branch>` which is invariant in the parent's working-tree state — uncommitted changes in the parent stay in the parent and are NOT copied into the new worktree. The subagent's worktree starts from the clean tip of the named branch (or HEAD if `BaseBranch == ""`). v1 documents this loudly: a subagent does NOT see the parent's WIP. Users who want WIP visibility must commit (or stash + apply manually) BEFORE dispatching the subagent. Auto-stash-on-dispatch is deferred to F15.5. - The subprocess child runs with `cmd.Dir = worktreePath`, so all relative paths the subagent uses are inside its own worktree. - On completion, the worktree is **left in place** (Q2=B: optional merge-back, not auto). Result map carries `worktree_path` so the user can inspect / merge / discard with the existing F04 `worktree cobra` commands.

5. Error handling, edge cases, and anti-bluff

5.1 Error paths

- **Max concurrency reached** — `Dispatch` returns `ErrMaxConcurrency` immediately (no goroutine, no worktree). Default cap: 5 (`opts.MaxConcurrency`). Slash / subagents list shows the running set so the user can `/subagents kill <id>` to free a slot.
- **Worktree creation failure** (e.g., dirty existing worktree, branch conflict) — `Dispatch` returns the wrapped `worktree.Manager.EnterWorktree` error. **Fail-closed**: do NOT silently degrade to in-process when the agent asked for worktree isolation. The agent receives the error and can retry.
- **WorktreeManager not configured but isolation=worktree requested** — `ErrWorktreeUnavailable`.
- **LLM provider failure** — propagated as `SubagentResult{State: StateFailed, Error: err.Error() }`. The channel is still closed normally. Tool result has `state: "failed"` and a non-empty error field.
- **Subprocess crash** (exit code $\neq 0$, stdout malformed) — manager constructs a `failedResult` with the captured stderr (truncated to 4 KiB) attached as `error`, and the exit code in `exit_code`. The result is still emitted on the channel.
- **Timeout** — context deadline propagates to the spawner. In-process: the inner agent loop sees `ctx.Err() == context.DeadlineExceeded` on its next LLM/tool call and returns. Subprocess: `exec.CommandContext` SIGKILLs the child. Manager emits `SubagentResult{State: StateTimedOut}`.
- **Kill via /subagents kill <id>** — manager calls `sub.cancel()`. In-process: `ctx.cancel` propagates. Subprocess: process group SIGTERM (then SIGKILL after 5s). Manager emits `SubagentResult{State: StateKilled}`.
- **Manager Close while subagents running** — `Close(ctx)` cancels every active subagent's `ctx` and waits for the goroutines to drain (or `ctx.Done()`). In-flight worktrees are NOT removed (the user keeps them).

- **Channel never closed bug** — manager’s dispatch goroutine ALWAYS closes the channel in a deferred call. Unit test asserts this with `_, ok := <-ch` after a known-failing dispatch.

5.2 Anti-bluff (CONST-035 / §11.9) — LOUD

The single largest bluff vector for F15 is “subagent ran” with no actual LLM call, no actual subprocess, no actual worktree — just a struct that “represents” a subagent but never executes. The criteria below are non-negotiable.

Required real-execution criteria (these define what “subagent ran” means in F15):

1. **In-process subagent** MUST actually invoke `llm.Provider.Generate(ctx, *LLMRequest)`. The test asserts: `FakeLLMProvider.GenerateCallCount() >= 1` after `WaitAll` returns. The subagent’s `Output` field MUST be the provider’s response, not the prompt echoed back. A “subagent” that returns `result.Output = task.Prompt` is a bluff and the test must fail.
2. **Subprocess subagent** MUST actually fork-exec the host binary. The test asserts: child PID was distinct from parent (`cmd.ProcessState.Pid() != os.Getpid()`); child’s `cmd.ProcessState.Success()` reflects the actual exit code; stdout JSON parses to a real `SubagentResult` with non-zero `Duration`.
3. **Worktree-isolated subagent** MUST actually create a worktree. The test asserts: `os.Stat(result.WorktreePath)` reports a directory; the path is NOT equal to the parent’s `cwd`; running `git rev-parse --show-toplevel` inside that path returns the path itself (not the parent repo). A subagent that creates a `result.WorktreePath` field but no actual directory is a bluff.
4. **Streaming aggregation** MUST actually stream — the channel MUST be unbuffered or `buffer-1`, and the test asserts that for two subagents A (fast) and B (slow), A’s result lands on the channel before B’s. A “streaming” implementation that collects all results in a slice and then sends them at once is a bluff.

The Challenge harness uses an in-tree `FakeLLMProvider` (a struct that implements `llm.Provider` and returns deterministic canned responses based on the prompt). The provider is explicitly documented as a **TEST PROVIDER, not a production mock** — the type docstring must read `// FakeLLMProvider is a TEST-ONLY llm.Provider used by F15's challenge harness and unit tests. It MUST NOT be referenced from production code paths (cmd/, applications/, internal/<pkg>/<file>.go that doesn't end in _test.go)`. Verified by the bluff scanner.. The `FakeLLMProvider` lives at `internal/agent/subagent/types.go` (NOT in a `_test.go` file) so the integration-test binary and the Challenge harness can both link it; the bluff scanner gets a special-case rule that `FakeLLMProvider` is the ONE allowed test-provider type and that it MUST carry the documented anti-misuse comment. Any other “fake provider” in production paths is a bluff.

FakeLLMProvider exact contract: - `Generate(ctx, req)` returns `&LLMResponse{Text: canned[req.Prompt]}` where `canned` is a `map[string]string` populated by the test/Challenge before dispatch. If `canned[req.Prompt]` is missing, returns `&LLMResponse{Text: "FAKE-LLM-ECHO: " + req.Prompt}` so test assertions can detect “provider was actually called with this prompt”. - `GenerateCallCount() int` returns the number of times `Generate` was invoked. - `LastPrompt() string` returns the most recent prompt body (assertion target). - `GetType()` returns `ProviderType("fake-test-only")` (a sentinel that production code never produces).

Optional real-LLM gated phase (mirrors F12): if `ANTHROPIC_API_KEY` is set in the env and not empty, the Challenge runs Phase D (a single in-process subagent against the real Anthropic provider, prompt "Reply with the literal string p1f15-real-llm-ok"). If the key is unset, Phase D is skipped with `SKIP-OK: P1-F15 ANTHROPIC_API_KEY unset (export ANTHROPIC_API_KEY=...)`. Phases A-C use only the `FakeLLMProvider`.

Concrete forbidden phrases (anti-bluff smoke):

```
cd HelixCode && grep -rn "simulated\\|for now\\|TODO implement\\|
placeholder" \
    internal/agent/subagent internal/tools/task_tool.go internal/
    commands/subagents_command.go \
    && echo BLUFF || echo clean
```

Must always print `clean`.

Helper-mode dispatch is the one early-exit the subprocess subagent path depends on; the in-process path does NOT depend on it. The Challenge MUST exercise BOTH paths to prove the architecture isn't a single point of bluff.

6. Testing

6.1 Unit (mocks OK)

- `TestIsolation_DefaultIsNone`.
- `TestSubagentTask_TimeoutDefaultsTo5Min`.
- `TestSubagentTask_TimeoutClampedTo30Min`.
- `TestFakeLLMProvider_RecordsCallCount + TestFakeLLMProvider_RecordsLastPrompt`.
- `TestFakeLLMProvider_HasAntiMisuseComment` — grep self-test against `types.go`.
- `TestInProcessSpawner_InvokesLLMProvider` — assert `FakeLLMProvider.GenerateCallCount() == 1` after `Spawn`.
- `TestInProcessSpawner_OutputIsLLMResponse_NotPromptEcho` — anti-bluff (b).
- `TestInProcessSpawner_PropagatesContextCancel`.
- `TestSubprocessSpawner_BuildsExecCommandWithSentinelEnv` — assert `cmd.Env` contains `HELIX_SUBAGENT_INVOCATION=1`.
- `TestSubprocessSpawner_SetsCwdToWorktreePath`.
- `TestSubprocessSpawner_DecodesStdoutJSON`.
- `TestManager_Dispatch_ReturnsIDAndChannel`.
- `TestManager_Dispatch_RespectsMaxConcurrency` — 6th dispatch with `cap=5` returns `ErrMaxConcurrency`.
- `TestManager_Dispatch_ChannelClosesEvenOnSpawnError` — channel must close even if spawner errors before emitting.
- `TestManager_Dispatch_StreamsResultsAsTheyComplete` — A finishes 50ms before B; A's result lands first. (Anti-bluff (4).)
- `TestManager_Kill_CancelsRunningSubagent`.
- `TestManager_Kill_UnknownID_ReturnsErrSubagentNotFound`.
- `TestManager_List_IncludesPendingAndRunning`.
- `TestManager_Close_DrainsRunningSubagents`.
- `TestManager_WaitAll_DrainsChannelToClose`.

- `TestTaskTool_Name_IsTask`.
- `TestTaskTool_IsolationDefaultsNone`.
- `TestTaskTool_RejectsInvalidIsolation`.
- `TestTaskTool_ResultMapShape` — keys: `id`, `state`, `output`, `duration_ms`, `tool_calls`, `isolation`, `worktree_path` (only when `isolation=worktree`).
- `TestSubagentsCommand_List_RendersTable`.
- `TestSubagentsCommand_Status_ShowsDescriptionNotPromptBody` — `CONST-042` anti-leak.
- `TestSubagentsCommand_Kill_DelegatesToManager`.
- `TestHelperMode_IsSubagentInvocation_ReadsEnvVar`.
- `TestHelperMode_RunAsSubagent_DecodesPayloadFromEnv`.
- `TestHelperMode_RunAsSubagent_PrintsJSONResult`.

6.2 Integration (-tags=integration)

- `TestSubagent_InProcess_RealLLM_RealToolRegistry` — uses `FakeLLMProvider`; runs against a `t.TempDir`-built `tools.ToolRegistry`. Asserts `GenerateCallCount == 1` and `result.Output` matches the canned response.
- `TestSubagent_Subprocess_RealForkExec_FakeLLM` — gated on `os.Executable()` being a real Go binary (always true in `go test`); runs the host binary with `HELIX_SUBAGENT_INVOCATION=1` and a payload that uses `FakeLLMProvider`. Asserts the child PID is distinct from `os.Getpid()` and the JSON stdout parses.
- `TestSubagent_Worktree_RealGitWorktree` — gated on `git` being on `PATH` (SKIP-OK: `P1-F15 git not on PATH (install: apt install git OR dnf install git)`). Initialises a `tempdir` repo with a single commit, dispatches a `worktree`-isolated subagent, asserts: (a) `result.WorktreePath` exists and is a directory, (b) `git rev-parse --show-toplevel` inside that path returns the path itself, (c) `os.Stat(result.WorktreePath)` succeeds.
- `TestSubagent_StreamingOrder` — dispatches two subagents with different fake LLM latencies (`FakeLLMProvider` grows a per-prompt sleep field); asserts the fast one's result lands on the channel first.
- `TestSubagent_Kill_Subprocess` — gated on `git`; dispatches a `worktree`-isolated subagent that runs a shell loop forever in its inner tool, calls `Kill(id)`, asserts result state is `StateKilled` within 6s.

6.3 Challenge

- **Phase A — In-process + FakeLLM (always runs)** — 5 [PASS] lines: dispatch returns `id+chan`; channel closes after `WaitAll`; `FakeLLM` was called; `result.Output` equals canned response (NOT prompt echo); `duration > 0`.
- **Phase B — Subprocess + FakeLLM (always runs)** — 4 [PASS] lines: child PID distinct from parent; stdout JSON decodes; `result.State=succeeded`; helper-mode env-var dispatch fired (proven by a sentinel printed to `stderr`).
- **Phase C — Worktree isolation + Subprocess + FakeLLM (gated on git)** — 4 [PASS] lines or [skipped: `git not on PATH (install: apt install git OR dnf install git)`]: `worktree` directory exists; `worktree path != parent cwd`; `git rev-parse --show-toplevel` inside path returns the path; `result.WorktreePath` populated.
- **Phase D — Real LLM (gated on ANTHROPIC_API_KEY)** — 2 [PASS] lines or [skipped: `ANTHROPIC_API_KEY unset (export`

ANTHROPIC_API_KEY=...)] : real provider returns non-empty text; text contains the literal p1f15-real-llm-ok.

- Final summary: IN-PROCESS=5/5 PASS; SUBPROCESS=4/4 PASS; WORKTREE=<n>/4 PASS; REAL-LLM=<n>/2 PASS.
- Challenge MUST exit non-zero on any assertion failure within phases that did run.

7. Cross-platform

Pure Go + stdlib (os/exec, syscall, os.Executable). No platform-specific code: - Subprocess re-exec works on Linux, macOS, Windows. - F04 worktree manager uses git worktree which is cross-platform. - The kill path uses cmd.Process.Signal(syscall.SIGTERM) on Unix and cmd.Process.Kill() on Windows (auto-selected by Go stdlib). - No build tags needed. The whole package builds on every GOOS.

8. Out of scope (deferred)

- **Auto-merge of subagent worktree changes back to the parent branch** — F15.5. v1: result map carries worktree_path; user uses F04's worktree cobra command for manual merge/discard.
- **Subagent-to-subagent direct messaging** (SendMessage tool from porting doc) — F15.5.
- **Per-subagent CPU/memory quotas** (cgroups v2 integration similar to F14's resource caps) — F15.5.
- **Persistent subagent state across CLI sessions** (resume a subagent after parent exit) — F15.5; would require a new persistence backend.
- **Subagent recursion / nested subagents** (a subagent itself calling task) — explicitly disabled in v1; the subprocess child does NOT register the task tool. F15.5 may lift this with a depth-limit guard.
- **subagent_type profile registry** (so prompts like subagent_type: "code-reviewer" resolve to a curated system prompt + tool subset) — v1 accepts the field but treats it as informational metadata only. F15.5 wires the actual profile resolver.
- **Auto-stash-on-dispatch** (copy parent's WIP into the worktree) — F15.5. v1 documents loudly that subagents start from the clean branch tip; users must commit/stash WIP first.
- **Streaming partial LLM output** (token-by-token from a subagent) — v1 streams only at the subagent-completion granularity. Token-level streaming is F15.5.

9. Constitutional compliance

- **§11.9 / CONST-035** — Challenge has FOUR phases (in-process, subprocess, worktree, real-LLM); the first two always run; the latter two are explicitly gated and never claim PASS without runtime evidence. Anti-bluff criteria 1-4 in §5.2 each have a dedicated unit/Challenge assertion. The FakeLLMProvider's anti-misuse comment is itself self-tested.
- **CONST-039** — Challenge at challenges/p1-f15-subagent-team/ + evidence harness at tests/integration/cmd/p1f15_challenge/main.go.
- **CONST-042 (No-Secret-Leak)** — SubagentTask.Prompt is logged at DEBUG level only; INFO-level logs use task.Description (which is human-supplied and safe). / subagents status prints description, never prompt body. Subprocess payload (env-var-encoded) is NEVER printed by the manager — only the description is logged when the subagent starts.
- **CONST-043 (No-Force-Push)** — close-out task pushes to all four remotes non-force.

- **No-Mocks-In-Production (Universal Rule 2)** — `SubagentManager`, both spawners, the `task` tool, the `/subagents` slash are real. The `FakeLLMProvider` is a TEST-ONLY type with a self-tested anti-misuse comment, and it is the SOLE exception (documented and bluff-scanner-allowlisted). Production code (`cmd/`, `applications/`) MUST NOT import it; the bluff scanner enforces this.

10. Open questions resolved

Q	Answer	Resolution
Q1: execution model	(C) hybrid	in-process goroutines by default; subprocess-per-subagent when <code>isolation: worktree</code> ; subprocess re-execs host binary with sentinel env var (mirrors F14 helper-mode)
Q2: worktree integration	(B) optional per task	<code>task</code> tool's <code>isolation</code> arg is "none" (default) or "worktree"; <code>v1</code> does NOT auto-merge — <code>worktree_path</code> returned in result for manual review via F04
Q3: results	(B) streaming	<code>Dispatch</code> returns <code><-chan SubagentResult</code> ; results land as each subagent completes; channel closes on terminal state; <code>WaitAll</code> helper drains synchronously
Q4: tool name	(B) <code>task</code>	matches claude-code precedent for cross-agent prompt portability; args: <code>description</code> , <code>prompt</code> , <code>isolation</code> (default "none"), <code>subagent_type</code> (optional)
Q5: user surface	(B) slash only	<code>/subagents</code> slash with <code>list/status/kill</code> ; no cobra subcommand